

Task 3 – Unit Testing & Logging

1. Objective

The objective of this task was to improve application reliability through automated testing and enhance maintainability by implementing structured logging.

This was achieved by:

- Writing unit tests for the service layer using JUnit 5 and Mockito.
 - Mocking repository dependencies to test business logic in isolation.
 - Configuring logging using SLF4J and Logback.
 - Recording important application events such as Create, Read, Update, and Delete operations.
-

2. Technologies Used

- Java 21
 - Spring Boot
 - Spring Data JPA
 - Maven
 - JUnit 5
 - Mockito
 - SLF4J
 - Logback
 - H2 Database
-

3. Task Requirements Mapping

Requirement	Implementation
Use JUnit for testing	Implemented JUnit 5 test cases
Use Mockito for mocking	Mocked ProductRepository dependency
Configure SLF4J / Logback	Added logger configuration and service logs
Include test reports	Maven Surefire reports generated

Requirement	Implementation
Demonstrate logging	Application logs captured during CRUD operations

4. Project Architecture

ProductService.java

Contains the business logic for:

- Creating products
- Retrieving all products
- Retrieving a product by ID
- Updating products
- Deleting products

Logging statements were added to track each operation.

```

1  package com.example.Product_api.service;
2
3  import com.example.Product_api.dto.ProductDTO;
4  import com.example.Product_api.model.Product;
5  import com.example.Product_api.repository.ProductRepository;
6  import org.springframework.stereotype.Service;
7
8  import java.util.List;
9  import java.util.Optional;
10
11 import org.slf4j.Logger;
12 import org.slf4j.LoggerFactory;
13
14 @Service
15 public class ProductService {
16
17     private static final Logger logger =
18         LoggerFactory.getLogger(ProductService.class);
19
20     private final ProductRepository repository;
21
22     public ProductService(ProductRepository repository) {
23         this.repository = repository;
24     }
25
26     // CREATE
27     public Product createProduct(ProductDTO dto) {
28
29         logger.info("Creating product: {}", dto.getName());
30
31         Product product =
32             new Product(
33                 dto.getName(),
34                 dto.getDescription(),
35                 dto.getPrice());
36
37         Product savedProduct = repository.save(product);
38
39         logger.info("Product created successfully with ID: {}", savedProduct.getId());

```

```

40
41     return savedProduct;
42 }
43
44 // READ ALL
45 public List<Product> getAllProducts() {
46
47     logger.info("Fetching all products");
48
49     return repository.findAll();
50 }
51
52 // READ ONE
53 public Optional<Product> getProductById(Long id) {
54
55     logger.info("Fetching product with ID: {}", id);
56
57     return repository.findById(id);
58 }
59
60 // UPDATE
61 public Optional<Product> updateProduct(Long id, ProductDTO dto) {
62
63     logger.info("Updating product with ID: {}", id);
64
65     return repository.findById(id)
66         .map(existingProduct -> {
67
68             existingProduct.setName(dto.getName());
69             existingProduct.setDescription(dto.getDescription());
70             existingProduct.setPrice(dto.getPrice());
71
72             logger.info("Product updated successfully: {}", id);
73
74             return repository.save(existingProduct);
75         });
76 }

```

```

78 // DELETE
79 public boolean deleteProduct(Long id) {
80
81     logger.info("Deleting product with ID: {}", id);
82
83     if (repository.existsById(id)) {
84
85         repository.deleteById(id);
86
87         logger.info("Product deleted successfully: {}", id);
88
89         return true;
90     }
91
92     logger.warn("Delete failed. Product not found: {}", id);
93
94     return false;
95 }
96 }

```

ProductServiceTest.java

Contains unit tests for ProductService.

Mockito is used to mock ProductRepository so that service methods can be tested independently without accessing the actual database.

```
src > test > java > com > example > Product_api > service > J ProductServiceTest.java > ...
1  package com.example.Product_api.service;
2
3  import com.example.Product_api.dto.ProductDTO;
4  import com.example.Product_api.model.Product;
5  import com.example.Product_api.repository.ProductRepository;
6
7  import org.junit.jupiter.api.Test;
8  import org.junit.jupiter.api.BeforeEach;
9  import org.mockito.InjectMocks;
10 import org.mockito.Mock;
11 import org.mockito.MockitoAnnotations;
12
13 import java.util.List;
14 import java.util.Optional;
15
16 import static org.junit.jupiter.api.Assertions.*;
17 import static org.mockito.Mockito.*;
18
19 class ProductServiceTest {
20
21     @Mock
22     private ProductRepository repository;
23
24     @InjectMocks
25     private ProductService service;
26
27     @BeforeEach
28     void setUp() {
29         MockitoAnnotations.openMocks(this);
30     }
31
32     @Test
33     void createProduct_ShouldSaveProduct() {
34
35         ProductDTO dto =
36             new ProductDTO(name: "Laptop",
37                             description: "Gaming Laptop",
38                             price: 1200.0);
39     }
```

```
src > test > java > com > example > Product_api > service > J ProductServiceTest.java > ...
```

```
19 class ProductServiceTest {
33 void createProduct_ShouldSaveProduct() {
40     Product savedProduct =
41         new Product(name: "Laptop",
42             description: "Gaming Laptop",
43             price: 1200.0);
44
45     when(repository.save(any(Product.class)))
46         .thenReturn(savedProduct);
47
48     Product result = service.createProduct(dto);
49
50     assertEquals("Laptop", result.getName());
51
52     verify(repository, times(1))
53         .save(any(Product.class));
54 }
55
56 @Test
57 void getAllProducts_ShouldReturnList() {
58
59     when(repository.findAll())
60         .thenReturn(List.of(
61             new Product(name: "Mouse",description: "RGB",price: 50.0),
62             new Product(name: "Keyboard",description: "Mechanical",price: 80.0)
63         ));
64
65     List<Product> products =
66         service.getAllProducts();
67
68     assertEquals(2, products.size());
69
70     verify(repository, times(1))
71         .findAll();
72 }
73
```

```
75 @Test
76 void getProductById_ShouldReturnProduct() {
77
78     Product product =
79         new Product(name: "Mouse",
80             description: "RGB",
81             price: 50.0);
82
83     when(repository.findById(1L))
84         .thenReturn(Optional.of(product));
85
86     Optional<Product> result =
87         service.getProductById(id: 1L);
88
89     assertTrue(result.isPresent());
90
91     verify(repository, times(1))
92         .findById(1L);
93 }
94
95 @Test
96 void updateProduct_ShouldUpdateExistingProduct() {
97
98     Product existingProduct =
99         new Product(name: "Laptop",
100             description: "Gaming Laptop",
101             price: 1200.0);
102
103     ProductDTO updatedDto =
104         new ProductDTO(name: "Laptop Pro",
105             description: "Updated Laptop",
106             price: 1500.0);
107
108     when(repository.findById(1L))
109         .thenReturn(Optional.of(existingProduct));
110
```

```

111         when(repository.save(any(Product.class)))
112             .thenReturn(invocation -> invocation.getArgument(0));
113
114         Optional<Product> result =
115             service.updateProduct(id: 1L, updatedDto);
116
117         assertTrue(result.isPresent());
118
119         assertEquals("Laptop Pro",
120             result.get().getName());
121
122         assertEquals("Updated Laptop",
123             result.get().getDescription());
124
125         assertEquals(1500.0,
126             result.get().getPrice());
127
128         verify(repository, times(1))
129             .findById(1L);
130
131         verify(repository, times(1))
132             .save(any(Product.class));
133
134
135     @Test
136     void deleteProduct_ShouldDeleteWhenExists() {
137
138         when(repository.existsById(1L))
139             .thenReturn(true);
140
141         boolean result =
142             service.deleteProduct(id: 1L);
143
144         assertTrue(result);
145
146         verify(repository, times(1))
147             .deleteById(1L);
148     }
149 }

```

ProductRepository.java

Provides database access operations through Spring Data JPA.

Repository methods are mocked during testing.

5. Logging Configuration

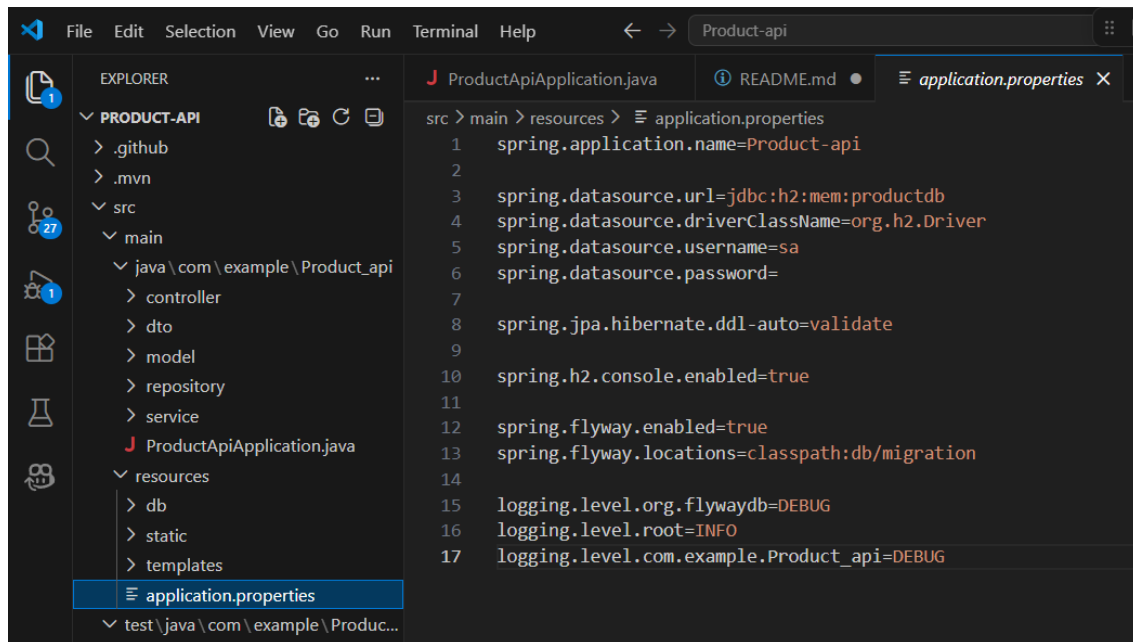
application.properties

The following logging configuration was added:

```
logging.level.org.flywaydb=DEBUG
logging.level.root=INFO
logging.level.com.example.Product_api=DEBUG
```

Purpose

- DEBUG logs are enabled for application-specific classes.
- INFO logs are enabled for general application events.
- Flyway migration logs remain visible for debugging.



6. Logging Implementation

The ProductService class uses SLF4J Logger.

Logger Initialization

```
private static final Logger logger =
    LoggerFactory.getLogger(ProductService.class);
```

Example Log Messages

```
logger.info("Fetching all products");
logger.info("Updating product with ID: {}", id);
logger.info("Product updated successfully: {}", id);
```

Benefits

- Tracks application activity.
- Simplifies debugging.

- Provides runtime visibility into service operations.
- Helps monitor API usage and behavior.

```
13
14 @Service
15 public class ProductService {
16
17     private static final Logger logger =
18         LoggerFactory.getLogger(ProductService.class);
19
20     private final ProductRepository repository;
21
```

7. Unit Testing Implementation

Test Framework

JUnit 5 was used for writing test cases.

Mockito was used to mock ProductRepository.

Features Tested

Create Product

Verifies that a product is saved successfully.

```
@Test
void createProduct_ShouldSaveProduct() {

    ProductDTO dto =
        new ProductDTO(name: "Laptop",
            description: "Gaming Laptop",
            price: 1200.0);

    Product savedProduct =
        new Product(name: "Laptop",
            description: "Gaming Laptop",
            price: 1200.0);

    when(repository.save(any(Product.class)))
        .thenReturn(savedProduct);

    Product result = service.createProduct(dto);

    assertEquals("Laptop", result.getName());

    verify(repository, times(1))
        .save(any(Product.class));
}
```


Get All Products

Verifies retrieval of all products.

```
@Test
void getAllProducts_ShouldReturnList() {

    when(repository.findAll())
        .thenReturn(List.of(
            new Product(name: "Mouse",description: "RGB",price: 50.0),
            new Product(name: "Keyboard",description: "Mechanical",price: 80.0)
        ));

    List<Product> products =
        service.getAllProducts();

    assertEquals(2, products.size());

    verify(repository, times(1))
        .findAll();
}
```

Get Product By ID

Verifies retrieval of a specific product.

```
@Test
void getProductById_ShouldReturnProduct() {

    Product product =
        new Product(name: "Mouse",
            description: "RGB",
            price: 50.0);

    when(repository.findById(1L))
        .thenReturn(Optional.of(product));

    Optional<Product> result =
        service.getProductById(id: 1L);

    assertTrue(result.isPresent());

    verify(repository, times(1))
        .findById(1L);
}
```

Update Product

Verifies successful modification of existing product details.

```
@Test
void updateProduct_ShouldUpdateExistingProduct() {

    Product existingProduct =
        new Product(name: "Laptop",
                    description: "Gaming Laptop",
                    price: 1200.0);

    ProductDTO updatedDto =
        new ProductDTO(name: "Laptop Pro",
                      description: "Updated Laptop",
                      price: 1500.0);

    when(repository.findById(1L))
        .thenReturn(Optional.of(existingProduct));

    when(repository.save(any(Product.class)))
        .thenAnswer(invocation -> invocation.getArgument(0));

    Optional<Product> result =
        service.updateProduct(id: 1L, updatedDto);

    assertTrue(result.isPresent());

    assertEquals("Laptop Pro",
                 result.get().getName());

    assertEquals("Updated Laptop",
                 result.get().getDescription());

    assertEquals(1500.0,
                 result.get().getPrice());

    verify(repository, times(1))
        .findById(1L);

    verify(repository, times(1))
        .save(any(Product.class));
}
```

Delete Product

Verifies successful deletion of products.

```
@Test
void deleteProduct_ShouldDeleteWhenExists() {

    when(repository.existsById(1L))
        .thenReturn(true);

    boolean result =
        service.deleteProduct(id: 1L);

    assertTrue(result);

    verify(repository, times(1))
        .deleteById(1L);
}
```

8. Test Execution

Unit tests were executed using Maven.

Command

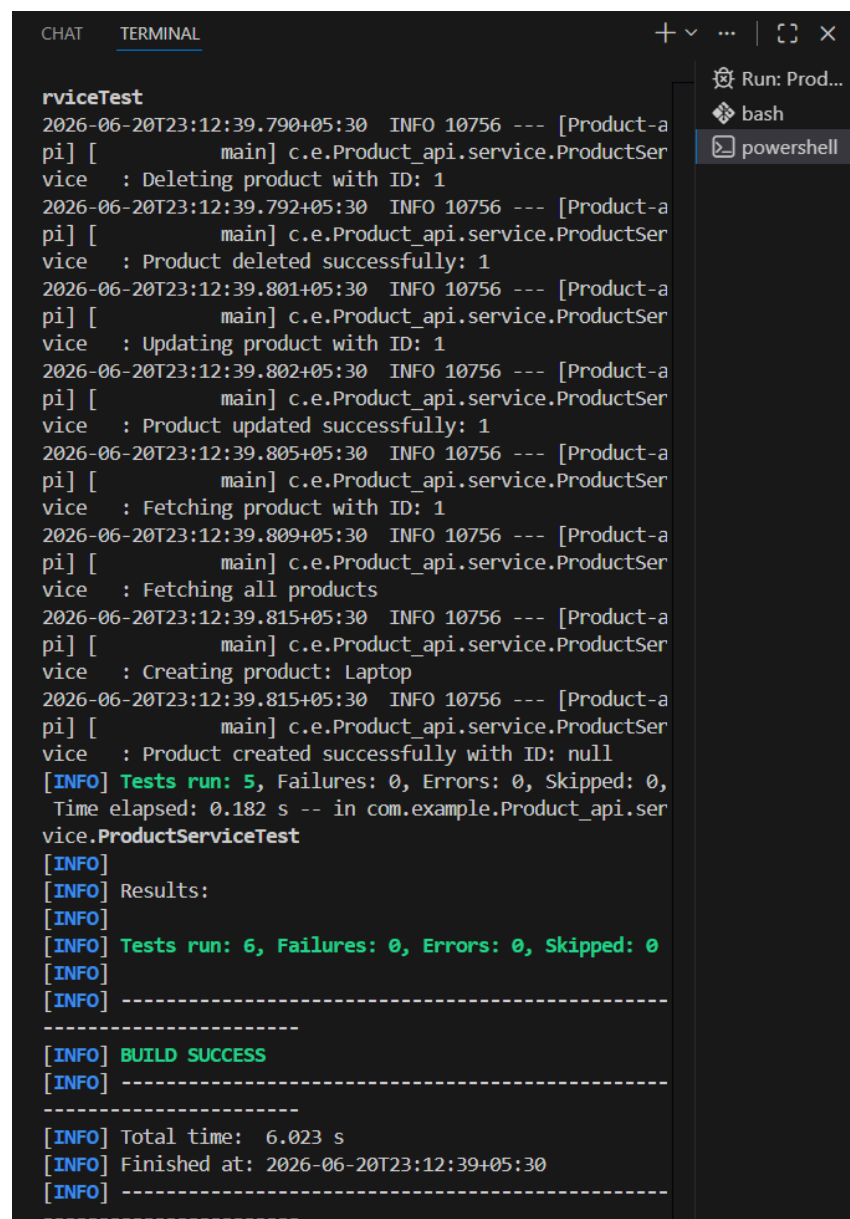
Git Bash

`./mvnw test`

Result

All tests executed successfully with:

BUILD SUCCESS



The screenshot shows a terminal window with a dark theme. The title bar includes 'CHAT' and 'TERMINAL' tabs. The terminal output displays the results of a Maven test execution. It shows several log messages with timestamps and log levels (INFO) indicating the deletion and updating of a product with ID 1, and the creation of a product named 'Laptop'. The output concludes with a summary of test results: 6 tests run, 0 failures, 0 errors, and 0 skipped. The final status is 'BUILD SUCCESS'. On the right side of the terminal, there is a vertical toolbar with icons for 'Run: Prod...', 'bash', and 'powershell'.

```
rvicetest
2026-06-20T23:12:39.790+05:30 INFO 10756 --- [Product-api] [main] c.e.Product_api.service.ProductService : Deleting product with ID: 1
2026-06-20T23:12:39.792+05:30 INFO 10756 --- [Product-api] [main] c.e.Product_api.service.ProductService : Product deleted successfully: 1
2026-06-20T23:12:39.801+05:30 INFO 10756 --- [Product-api] [main] c.e.Product_api.service.ProductService : Updating product with ID: 1
2026-06-20T23:12:39.802+05:30 INFO 10756 --- [Product-api] [main] c.e.Product_api.service.ProductService : Product updated successfully: 1
2026-06-20T23:12:39.805+05:30 INFO 10756 --- [Product-api] [main] c.e.Product_api.service.ProductService : Fetching product with ID: 1
2026-06-20T23:12:39.809+05:30 INFO 10756 --- [Product-api] [main] c.e.Product_api.service.ProductService : Fetching all products
2026-06-20T23:12:39.815+05:30 INFO 10756 --- [Product-api] [main] c.e.Product_api.service.ProductService : Creating product: Laptop
2026-06-20T23:12:39.815+05:30 INFO 10756 --- [Product-api] [main] c.e.Product_api.service.ProductService : Product created successfully with ID: null
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.182 s -- in com.example.Product_api.service.ProductServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO]
[INFO] Total time: 6.023 s
[INFO] Finished at: 2026-06-20T23:12:39+05:30
[INFO] -----
```

9. Test Reports

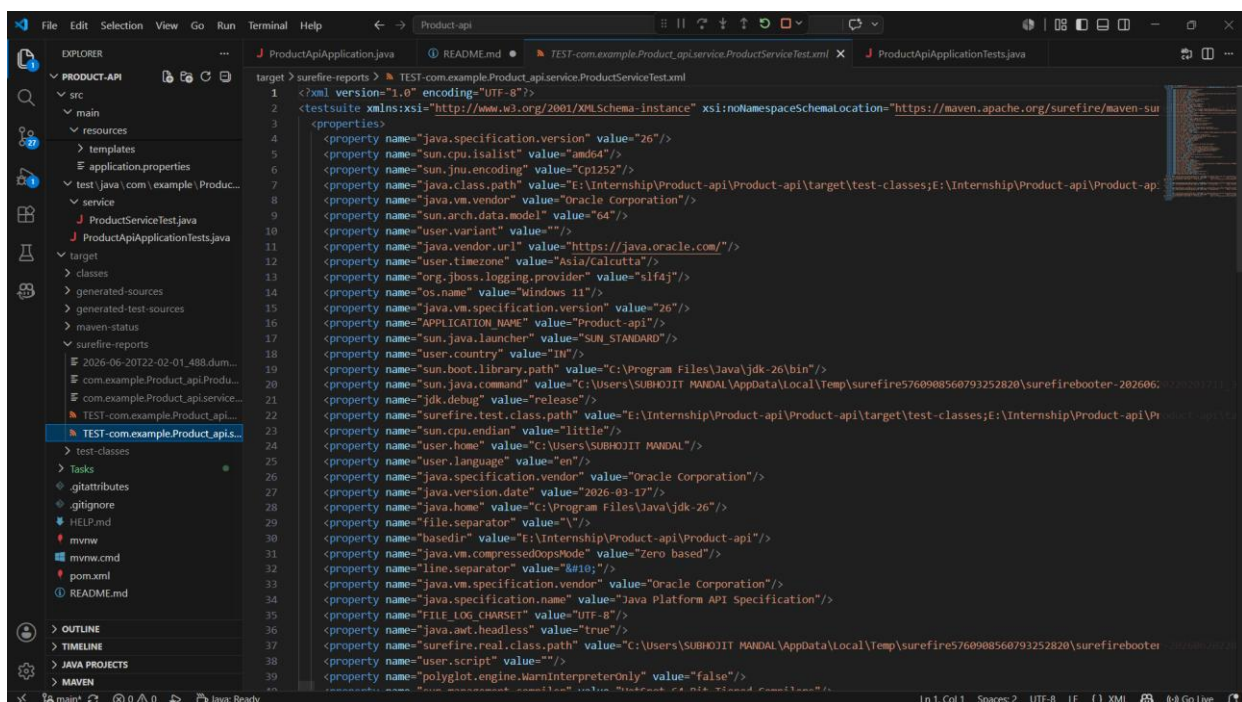
Maven Surefire automatically generated test reports.

Location

target/surefire-reports

Purpose

- Stores detailed test execution results.
- Records passed and failed test cases.
- Useful for debugging and verification.



10. Logging Verification

The application was started and API requests were executed.

Example Request

curl http://localhost:8080/api/products

Corresponding Log

Fetching all products

Example Update Request

```
curl -X PUT http://localhost:8080/api/products/1 \  
-H "Content-Type: application/json" \  
-d '{"name":"Android","description":"Android Phone","price":15000}'
```

Corresponding Log

Updating product with ID: 1
Product updated successfully: 1

CURL Commands:

```
SUBHOJIT MANDAL@SUBHOJIT MINGW64 /e/Internship/Product-api/Product-api (main)  
● $ curl http://localhost:8080/api/products  
[{"id":1,"name":"Gaming Mouse","description":"Wireless RGB mouse","price":490.99}, {"id":2,"name":"Mechanical Keyboard","description":"Cherry MX Blue switches","price":890.5}, {"id":3,"name":"4K Monitor","description":"32-inch ultra HD display","price":34900.0}]  
SUBHOJIT MANDAL@SUBHOJIT MINGW64 /e/Internship/Product-api/Product-api (main)  
$ curl -X PUT http://localhost:8080/api/products/1 \  
-H "Content-Type: application/json" \  
-d '{"name":"Android","description":"Android Phone","price":15000}'  
● {"id":1,"name":"Android","description":"Android Phone","price":15000.0}
```

Logging Output Generated During API Operations:

```
2026-06-20T22:07:43.526+05:30 INFO 24652 --- [Product-api] [nio-8080-exec-1] c.e.Product_api.service.ProductService : Fetching all products  
2026-06-20T22:07:54.146+05:30 INFO 24652 --- [Product-api] [nio-8080-exec-2] c.e.Product_api.service.ProductService : Updating product with ID: 1  
2026-06-20T22:07:54.163+05:30 INFO 24652 --- [Product-api] [nio-8080-exec-2] c.e.Product_api.service.ProductService : Product updated successfully: 1  
□
```

11. Deliverables Completed

Unit Testing

- Implemented service layer unit tests.
- Mocked repository dependencies using Mockito.
- Verified business logic independently.

Logging

- Configured SLF4J and Logback.
- Added logging statements for CRUD operations.
- Verified log generation during API execution.

Test Reports

- Generated Maven Surefire reports.
 - Verified successful execution of test cases.
-

12. Conclusion

Task 3 was successfully completed by implementing unit testing and logging in the Spring Boot Product API.

JUnit 5 and Mockito were used to validate service-layer functionality, while SLF4J and Logback were used to provide structured application logs. Test execution results and generated reports confirmed the correctness of the implementation, improving the reliability, maintainability, and debuggability of the application.

Linkedin Task3 Post Link: [Testing and Logging Task-3](#)

Github Repo Link: [subhojit1707-crypto/InternSpark-Internship](#)